



Art Institute Explorer - v2.0

Un buscador inteligente de obras de arte que permite a los usuarios explorar contenido del Art Institute of Chicago con autenticación, obra de arte sorpresa diaria y gestión de estado global avanzada.



Estado Actual - v2.0

Rama: v2_ArtApp

Fase Completada: ☒ Fase 5 - Obra de Arte Diaria en Modal

Fase Actual:  Fase 6 - Migración Final



Objetivos v2.0

La versión 2.0 introduce mejoras arquitectónicas y nuevas funcionalidades diseñadas para un proyecto de nivel **Junior/Aprendiz**:

Mejoras Técnicas

- ☒ **Estado Global con Redux** - Centralizar estado con Redux Toolkit
- ☒ **Redux DevTools** - Debugging avanzado del estado
- ☒ **Error Boundaries** - Captura y manejo de errores en componentes
- ☒ **React Hooks** - Uso correcto de useState, useEffect, useCallback, useRef
- ☒ **React Hook Form** - Validación de formularios simplificada

Nuevas Funcionalidades

- ☒ **Sistema de Autenticación** - Registro e inicio de sesión con Modal Overlay
-  **Obra de Arte Diaria** - Cada usuario logueado recibe una obra sorpresa al día
-  **Optimización de Imágenes** - Lazy loading y múltiples resoluciones
- ☒ **Rutas Protegidas** - Acceso limitado a usuarios autenticados

PROF



Características v1.0 (Mantienen)

- **Búsqueda Semántica:** Busca obras de arte por términos, artistas, técnicas y períodos
- **Filtros Dinámicos Avanzados:**
 - Búsqueda por palabras clave
 - Filtrado por artista
 - Filtrado por material/técnica
 - Rango de fechas (año de creación)
 - Filtrado por cultura
- **Galería Visual Fluida:** Interfaz responsive con animaciones suaves
- **Vista Detallada:** Modal con información completa de cada obra
- **Paginación Inteligente:** Navegación fácil entre páginas de resultados

- **Diseño Responsive:** Funciona perfectamente en desktop, tablet y móvil

Requisitos Previos

- Node.js 16+
- npm o yarn
- Conexión a Internet (para consumir la API del Art Institute)

Instalación

1. Clonar el repositorio

```
git clone https://github.com/AnaLauDB/Art-API.git
cd Art-API
```

2. Instalar dependencias

```
npm install
```

3. Iniciar el servidor de desarrollo

```
npm run dev
```

La aplicación estará disponible en <http://localhost:5173>

Estructura del Proyecto v2.0

```
src/
├── components/
│   ├── ErrorBoundary.jsx           # [FASE 2] Captura global de
errores
│   ├── DailyPickErrorBoundary.jsx  # [FASE 2] Error boundary
regional
│   ├── GalleryErrorBoundary.jsx    # [FASE 2] Error boundary
regional
│   ├── Auth/                      # [FASE 3] Componentes de
autenticación
│   │   ├── LoginForm.jsx           # Validación con React Hook Form
│   │   ├── RegisterForm.jsx       # Validación con React Hook Form
│   │   ├── ProtectedRoute.jsx     # Componente protegido
│   │   └── AuthModal.jsx          # Modal overlay flotante
│   └── Gallery/                   # Componentes de galería
(Mejorados v2.0)
│   └── ArtworkCard.jsx            # [FASE 4] Con lazy loading de
```

```

imágenes
|   |   |— ArtworkDetail.jsx
|   |   |— ArtworkGrid.jsx
|   |— DailyArtworkModal.jsx      # [FASE 5] Obra sorpresa diaria
en MODAL
|   |— OptimizedImage.jsx        # [FASE 4] Imagen con
Intersection Observer
|   |— SearchBar.jsx             # v1.0 (se migrará)
|   |— FilterPanel.jsx           # v1.0 (se migrará)
|   |— ArtworkCard.jsx           # v1.0 (se migrará) - ACTUALIZADO
FASE 4
|   |— ArtworkGrid.jsx           # v1.0 (se migrará)
|   |— ArtworkDetail.jsx         # v1.0 (se migrará)
|
|— services/
|   |— arteServices.js           # Llamadas API (Mantiene)
|   |— authServices.js          # [FASE 3] Lógica de
autenticación
|   |— imageServices.js          # [FASE 4] Optimización de
imágenes
|
|— redux/                        # ✅ [FASE 1] COMPLETADO
|   |— store.js
|   |— slices/
|       |— authSlice.js          # Estado de autenticación
|       |— artworksSlice.js      # Estado de obras de arte
|       |— dailyPickSlice.js     # Estado de obra diaria
|       |— filterSlice.js        # Estado de filtros
|       |— uiSlice.js            # Estado de UI (modales,
notificaciones)
|
|— utils/
|   |— constants.js             # URLs, configuraciones
|   |— dateUtils.js             # [FASE 5] Funciones de fecha
para obra diaria
|
|— styles/
|   |— SearchBar.css
|   |— FilterPanel.css
|   |— ArtworkGrid.css
|   |— ArtworkDetail.css
|   |— ArtworkCard.css          # ACTUALIZADO FASE 4
|   |— AuthModal.css            # [FASE 3] Estilos del modal
overlay
|   |— DailyArtwork.css          # [FASE 5] Estilos de obra diaria
|   |— variables.css             # CSS variables globales
|
|— App.jsx                      # [FASE 6] Se migrará a Redux
|— App.css
|— index.css

#### Resumen de Fases
- Fase 1 – Redux setup: ✅ Completada
- Fase 2 – Error boundaries: ✅ Completada

```

- Fase 3 – Autenticación (modal):  Completada
- Fase 4 – Optimización de imágenes:  Completada
- Fase 5 – Obra diaria (modal):  Completada
- Fase 6 – Migración a Redux + thunks:  Completada
- Detección de cambio de día
- Modal overlay con fixed positioning
- Renderizado condicional basado en autenticación
- Manejo de fechas en JavaScript
- Estado persistente entre recargas de página
- Z-index management en modales

Fase 6 - Migración Final

- ****useCallback**** - Optimizar re-renders
- ****useRef**** - Acceso directo al DOM
- ****Thunks**** - Acciones asincrónicas en Redux
- ****Testing**** - Validar componentes

****Aprendizajes:****

- Cuándo usar cada hook
- Performance optimization
- Testing en React

****Art Institute of Chicago API v1****

- Documentación: <https://api.artic.edu/docs/>
- Base URL: ``https://api.artic.edu/api/v1``

Endpoints Principales

- ``/artworks/search`` - Búsqueda de obras con filtros
- ``/artworks/{id}`` - Detalles de una obra específica
- ``/artists`` - Lista de artistas
- ``/artworks/search?facets=medium_display`` - Agregaciones de medios

Uso

Búsqueda Simple

1. Ingresa un término en la barra de búsqueda (ej: "paisaje", "Van Gogh", "óleo")
2. Haz clic en "Buscar" o presiona Enter
3. Los resultados se mostrarán en la galería

Filtros Avanzados

1. Usa el panel de filtros en la izquierda para aplicar criterios específicos
2. Combina múltiples filtros para refinar resultados
3. Haz clic en "Aplicar filtros" para actualizar la búsqueda
4. Usa "Limpiar filtros" para restablecer

Ver Detalles

1. Haz clic en cualquier obra de arte en la galería

2. Se abrirá un modal con información completa
3. Haz clic en "Ver en el Art Institute" para más información
4. Cierra el modal con el botón X

🎨 Tecnologías Utilizadas v2.0

Core

- **React 19.2.5** - Librería de UI
- **Vite 8.0.10** - Build tool y dev server
- **JavaScript ES6+** - Lenguaje de programación

Estado Global

- **Redux Toolkit** ✅ - Gestión de estado simplificada
- **React-Redux** ✅ - Integración React + Redux
- **@redux-devtools/extension** ✅ - DevTools para debugging

Autenticación y Formularios

- **React Hook Form** ✅ - Validación de formularios [Fase 3]
- **localStorage** ✅ - Persistencia de sesión [Fase 3]

Peticiones HTTP

- **Axios 1.15.2** - Cliente HTTP

Estilos

- **CSS3** - Grid, Flexbox, Animaciones
- **CSS Variables** - Temas y configuración global

Herramientas de Desarrollo

- **ESLint** - Linting de código
- **Redux DevTools Chrome Extension** - Debugging de estado

📋 Instalación y Setup v2.0

1. Clonar y dependencias

```
```bash
git clone https://github.com/AnaLauDB/Art-API.git
cd Art-API
npm install
```

PROF

## 2. Extensión Redux DevTools (Recomendado)

Para debuggear Redux en tiempo real, instala la extensión:

- **Chrome:** [Redux DevTools](#)
- **Firefox:** [Redux DevTools](#)

## 3. Iniciar desarrollo

```
npm run dev
```

La aplicación estará en <http://localhost:5174>

## 4. Validar Redux

Una vez iniciada:

1. Abre DevTools (F12)
2. Ve a la pestaña **Redux** (si instalaste la extensión)
3. Deberías ver las actions y el estado global



## Scripts Disponibles

```
Desarrollo
npm run dev # Inicia servidor de desarrollo

Producción
npm run build # Compilar para producción
npm run preview # Previsualizar build

Calidad de código
npm run lint # Ejecutar ESLint
```

## Características de Autenticación

- **Login/Registro:** Formularios validados con React Hook Form
- **Modal Overlay:** Interfaz flotante centrada en la pantalla con backdrop oscuro
- **Animaciones:** Transiciones suaves (fade in + slide up)
- **Toggle:** Cambiar entre login y registro sin perder datos
- **Persistencia:** Token guardado en localStorage
- **Rutas Protegidas:** Acceso limitado a usuarios logueados
- **Obra Diaria:** Cada usuario recibe una obra sorpresa al ingresar
- **Estado Global:** Redux para mantener datos de sesión
- **UX Mejorada:** Modal centrado en pantalla vs al final de página



## Responsive Design

La aplicación es totalmente responsive:

- **Desktop:** Barra lateral + galería principal
- **Tablet:** Layout adaptado (600px+)
- **Mobile:** Stack vertical optimizado (480px+)



## Ejemplos de Búsqueda

- **Artista:** "Pablo Picasso", "Leonardo da Vinci"
- **Técnica:** "óleo", "escultura", "fotografía"

- **Tema:** "paisaje", "retrato", "naturaleza muerta"
- **Período:** Combina años (ej: 1890-1900)

## Características API

### Búsqueda Semántica

```
// Buscar con términos combinados
await searchArtworks("impressionism painting");

// Con filtros avanzados
await getArtworksByFilters({
 query: "portrait",
 artist: "Rembrandt",
 medium: "oil",
 yearFrom: "1630",
 yearTo: "1669"
});
```

### Gestión de Imágenes

```
// URLs de imágenes optimizadas
getImageUrl(imageId, "small") // 150x150
getImageUrl(imageId, "medium") // 400x400
getImageUrl(imageId, "large") // 843x843
```

## Configuración

### Variables de Entorno

No requiere configuración especial. La API es pública y accesible sin autenticación.

### Build

El proyecto usa Vite con React. La configuración está en `vite.config.js`.

## Objetivos v2.0 - En Desarrollo

### Completar en Fase 2-6

-  Redux Setup y State Management
-  Error Boundaries (Global, Regional, Local)
-  Autenticación con Login/Registro (Modal Overlay)
-  Optimización de Imágenes con Lazy Loading (Fase 4 - Completada)
-  Sistema de Obra de Arte Diaria en Modal (Fase 5 - Completada)
-  Migración completa a Redux

- ☐ Testing de componentes

## Funcionalidades Adicionales

- ☒ Guardar obras favoritas en localStorage/Redux
- ☒ Compartir obras en redes sociales
- ☐ Modo oscuro/claro

## Documentación de Desarrollo

### Para Colaboradores

Esta es una aplicación de **nivel Junior/Aprendiz** diseñada para:

- Aprender Redux y manejo de estado global
- Comprender React Hooks profundamente
- Implementar Error Boundaries
- Trabajar con APIs externas
- Optimizar performance en galerías grandes

### Convenciones de Código

- **Componentes:** PascalCase
- **Archivos:** PascalCase (componentes), camelCase (services/utils)
- **Redux:** Usar Redux Toolkit con createSlice
- **Comentarios:** Explicar el "por qué", no el "qué"

### Debugging

1. **Redux DevTools:** Inspecciona el estado en tiempo real
2. **React DevTools:** Valida renderizaciones innecesarias
3. **Console:** Logs de errores y seguimiento
4. **Network:** Monitorea llamadas a la API

---

PROF

Este proyecto utiliza datos del Art Institute of Chicago bajo su política de Open Access.

## Contribuciones

Las contribuciones son bienvenidas. Por favor:

1. Fork el repositorio
2. Crea una rama para tu feature (`git checkout -b feature/AmazingFeature`)
3. Commit tus cambios (`git commit -m 'Add some AmazingFeature'`)
4. Push a la rama (`git push origin feature/AmazingFeature`)
5. Abre un Pull Request

## Autor

Ana Lau



- GitHub: [@AnaLauDB](#)